

TreeMAPPO: Fine-Grained Credit Assignment for Agentic LLM Training via Token-Level Trajectory Branching

Anonymous submission

Abstract

We present TreeMAPPO, a reference-free framework for fine-grained credit assignment in agentic large language model training. Rather than assigning a single scalar reward to an entire reasoning trajectory, TreeMAPPO constructs a rollout tree, estimates segment-level contribution and uncertainty from mixed-success leaves, and converts these estimates into dense token-level training signals. The method is designed for mathematical reasoning agents whose trajectories may interleave natural-language reasoning, fenced Python tool calls, tool responses, and final boxed answers. Compared with methods such as TEMPO that exploit shared prefixes among sampled trajectories, TreeMAPPO actively creates new continuations through guided token-level branching, concentrating exploration at uncertain and model-plausible decision points. Compared with TreeRPO-style tree credit assignment based on empirical outcome comparisons, TreeMAPPO uses a statistically grounded maximum-a-posteriori posterior model that estimates both signed segment contribution and uncertainty before producing token-level advantages. Experiments use a resource-efficient split pipeline that rolls out from the base policy, generates advantage-weighted training data, fine-tunes LoRA adapters, and validates per-epoch checkpoints, comparing TreeMAPPO against base models, trajectory-level GRPO-style training, and tree-structured ablations across tool and no-tool mathematical reasoning settings.

Introduction

Sparse end-of-trajectory supervision makes credit assignment a central bottleneck in reinforcement learning for reasoning agents. When a long rollout contains many reasoning tokens, optional tool calls, and only a terminal correctness judgment, a single trajectory-level reward provides little guidance about which intermediate decisions should be reinforced or suppressed. The difficulty is especially acute in agentic settings, where early errors may be repaired by later reasoning or external tool feedback.

Recent work has begun to exploit shared-prefix and tree structure to densify supervision, including implicit prefix-tree methods, explicit tree-sampling methods, off-policy prefix-aware methods, and fine-grained agentic branching methods (Tran, Yao, and Yu 2025; Yang et al. 2025; Li et al. 2025; Huang, Nguyen, and Zimmer 2025; Wang et al. 2026). These methods show that branching structure exposes local

comparisons that are unavailable to flat rollout groups. However, existing approaches often rely on naturally occurring prefix overlap, fixed segment boundaries, offline teacher-generated trees, or heuristic credit estimators that do not explicitly represent uncertainty.

This paper develops fine-grained credit assignment for mathematical reasoning agents through token-level trajectory branching. The central idea is to turn a small set of full rollouts into a structured tree of alternative continuations, then infer which segments are helpful, harmful, or uncertain from the pattern of correct and incorrect descendants. We call the resulting method TreeMAPPO (Tree-based Maximum-A-Posteriori Policy Optimization).

Our implementation uses a lightweight tool-calling protocol: the model receives a question and an optional Python tool specification, emits reasoning in text, may invoke the tool via fenced Python markdown blocks, observes the tool response in context, and is required to terminate with a final answer wrapped in `\boxed{}`. Rollouts are organized into trees, branch points are chosen at token granularity, and segment-level posterior estimates are transformed into token-level training advantages. The experiments in this submission emphasize a split one-shot pipeline: collect rollouts from the base model, construct the training set once for each condition, train adapters for multiple epochs, and validate the resulting checkpoints. This isolates the proposed credit-assignment mechanism while keeping the compute budget tractable.

Our contributions are:

- We formulate fine-grained credit assignment as posterior inference over signed segment contributions in a trajectory tree.
- We introduce a guided branching policy that prioritizes uncertain segments and branching nodes while discouraging over-fragmentation and over-concentration.
- We describe a practical training pipeline that converts rollout trees into non-redundant training trajectories with token-level advantages.
- We provide an experimental framework spanning tool and no-tool reasoning, in-distribution and out-of-distribution mathematical benchmarks, and ablations that separate branching strategy from credit estimation.

The TreeMAPPO (Tree Maximum-A-Posteriori) Algorithm

Q: What is $3 \times 2 + 1$?

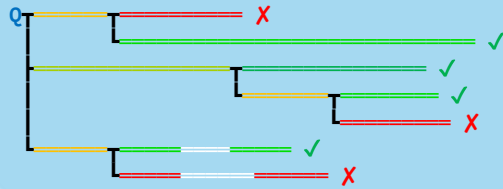
1 Generate the trunk trajectories.

Q $3 \times 2 = 42$. $42 + 1 = \boxed{43}$ ✗
 Let's do it step by step. $3 \times 2 + 1 = 6 + 1 = \boxed{7}$ ✓
 python `3*2+1` `7` `\boxed{7}` ✓

2 Branch from existing trunks.

Q $3 \times 2 = 42$. $42 + 1 = \boxed{43}$ ✗
 wait, actually $3 \times 2 = 6$. $6 + 1 = \boxed{7}$ ✓
 Let's do it step by step. $3 \times 2 + 1 = 6 + 1 = \boxed{7}$ ✓
 $3 \times 2 = 6$. $6 + 1 = \boxed{7}$ ✓
 $6 + 1 = \boxed{42}$ ✗
 python `3*2+1` `7` `\boxed{7}` ✓
`x2+1` `error` `\boxed{error}` ✗

3 Calculate the advantages using Maximum-A-Posteriori algorithm and our proposed segment contribution model.



4 Generate the flattened training trajectories from the tree with segment-level credit assignment.

Q $3 \times 2 = 42$. $42 + 1 = \boxed{43}$
 Q $3 \times 2 = 42$. wait, actually $3 \times 2 = 6$. $6 + 1 = \boxed{7}$
 Q Let's step. $3 \times 2 + 1 = 6 + 1 = \boxed{7}$
 Q Let's step. $3 \times 2 = 6$. $6 + 1 = \boxed{7}$
 Q Let's step. $3 \times 2 = 6$. $6 + 1 = \boxed{42}$
 Q python `3*2+1` `7` `\boxed{7}`
 Q python `3x2+1` `error` `\boxed{error}`

Figure 1: Method overview. TreeMAPPO samples base-policy trajectories, expands selected token-level branch points, infers segment-level contribution and uncertainty from leaf correctness, and converts the resulting posterior estimates into token-level advantages for adapter training.

Related Work

Trajectory-level RL for reasoning. Reinforcement learning with verifiable rewards has become a standard recipe for improving mathematical and agentic reasoning, because final-answer correctness can often be checked automatically. PPO remains a canonical policy-optimization baseline; it stabilizes policy updates with a clipped surrogate objective and obtains token-level advantages through a learned value function (Schulman et al. 2017). GRPO, introduced in DeepSeekMath, removes the critic by normalizing outcome rewards within groups of sampled responses, substantially reducing training complexity while retaining the appeal of verifiable rewards (Shao et al. 2024). The limitation of both trajectory-level and group-relative outcome supervision is the one targeted here: when a long response receives only terminal feedback, many irrelevant tokens are reinforced or suppressed together with the decisive reasoning steps.

Process supervision and verifier-based credit. A complementary line of work makes supervision denser by judging intermediate reasoning steps. Let’s Verify Step by Step showed that process supervision can outperform outcome-only supervision on mathematical reasoning by training process reward models from step-level human labels (Lightman et al. 2023). This line establishes the value of fine-grained credit, but it changes the supervision regime: it depends on annotated intermediate labels, learned verifiers, or ad-

ditional reward-model infrastructure. TreeMAPPO instead keeps the outcome-only setting and infers local credit from the structure of sampled rollouts. The distinction is important methodologically: our goal is not to build a stronger verifier, but to convert sparse terminal correctness into localized training signals without step-level references.

Implicit tree and shared-prefix methods. TEMPO (Tran, Yao, and Yu 2025) converts a group of sampled responses into a prefix tree and adds branch-gated temporal-difference corrections on top of a GRPO-style training loop. Its main appeal is that it preserves a relatively simple rollout interface while extracting token-level signal where natural branching occurs. Our method differs in that we do not rely on incidental prefix overlap alone; instead, we explicitly create new branches at selected token positions, which provides direct control over branching density and lets exploration be driven by posterior uncertainty.

Explicit tree sampling and tree-aware advantages. TreeRPO (Yang et al. 2025) and TreePO (Li et al. 2025) both pursue explicit tree-structured sampling to obtain denser supervision. TreeRPO estimates step-level reward expectations via tree sampling and forms step-level comparison groups without a separate reward model. TreePO emphasizes systems efficiency, shared-prefix reuse, and subgroup-relative advantages over segment-level trees. Our method is closest in spirit to this line, but differs in two impor-

tant respects: first, we branch at arbitrary token positions rather than imposing fixed step or segment partitions; second, we infer segment contribution with a probabilistic MAP model that explicitly represents uncertainty, rather than relying only on win-rate or subgroup heuristics.

Prefix-aware and off-policy tree supervision. Tree-OPO (Huang, Nguyen, and Zimmer 2025) uses teacher-generated MCTS prefixes and staged advantage estimation to create prefix-aware off-policy training groups. By contrast, TreeMAPPO is on-policy and reference-free: the tree is constructed directly from the policy’s own rollouts, without offline teacher trees, reward models, or process-level supervision.

Agentic RL and branching granularity. APPO (Wang et al. 2026) studies branching and credit assignment in tool-using agents, arguing that influential decision points are not confined to tool-call boundaries or coarse workflow stages. This observation aligns closely with our setting. The main methodological difference is that APPO uses a branching score and procedure-level advantage scaling, whereas TreeMAPPO combines explicit token-level branching with posterior inference over signed segment contribution and uncertainty.

Tool-augmented mathematical reasoning and benchmarks. Our agent protocol is also related to work on tool-augmented reasoning. PAL and Program-of-Thoughts showed that language models can delegate arithmetic or symbolic computation to Python programs, improving numerical reasoning by separating decomposition from execution (Gao et al. 2022; Chen et al. 2022). ReAct generalized interleaved reasoning and acting, while Toolformer studied how models can learn tool use from limited demonstrations (Yao et al. 2022; Schick et al. 2023). These methods motivate the mixed text–tool trajectories considered here, but they primarily address inference-time tool use rather than training-time credit assignment. Our evaluation setting builds on mathematical reasoning resources such as MATH (Hendrycks et al. 2021) and recent verifiable training corpora such as DeepMath-103K (He et al. 2025); the broader benchmark mix is intended to test whether tree-based credit assignment transfers beyond the distribution used to construct training trajectories.

Problem Setting and Agent Protocol

We study question-answering agents for mathematical reasoning. Each sample provides a problem statement and a ground-truth final answer used only for post hoc correctness judgment. During rollout, the model may produce plain-text reasoning and, when enabled, invoke a Python executor.

Our implementation uses the following protocol.

1. Each prompt contains the problem statement and, when tool use is enabled, a specification that Python can be called by emitting a fenced markdown block.
2. The model may interleave reasoning and tool use.
3. When a fenced Python block closes, generation pauses, the tool executes, and the tool response is appended to the context.

4. The model is instructed to present its final answer in `\boxed{}`.
5. If generation reaches a control handoff without a boxed answer, the framework requests continuation until a boxed answer is produced or rollout termination conditions are hit.

This protocol is consistent across training-data collection and evaluation so that differences across methods are attributable to branching and credit assignment rather than prompt format changes.

Method

Overview

For each question, we first sample a set of complete trunk trajectories. We then expand a rollout tree by branching from selected token positions. Each completed leaf trajectory is judged as correct or incorrect using its extracted final boxed answer. The resulting leaf labels provide supervision for segment-level posterior inference. Posterior estimates are transformed into branching scores for future expansions and into training advantages for model optimization.

Trajectory Tree Construction

In the default configuration for our main tool-using setup, we begin with 4 trunk trajectories and expand to 16 total trajectories. Our experimental plan also includes branch-budget variants with 8 and 32 total trajectories.

Branching is token-level rather than step-level. A new branch can be created in two ways:

- **Segment split:** split an existing reasoning segment at an internal token position and regenerate from that point.
- **Node expansion:** branch again from an existing branching node, increasing its branching factor.

The implementation keeps a fixed decoding temperature within each tree. In the current configs, this value is $T = 0.7$ for trunk generation, continuation, and branch expansion.

MAP Credit Assignment Model

Let ℓ index judged leaves and let i index non-prompt segments. For each leaf ℓ :

- $y_\ell \in \{+1, -1\}$ is the correctness label.
- $x_{\ell,i} \in \{0, 1\}$ indicates whether segment i lies on the path to leaf ℓ .
- Segment i has mean contribution m_i and log standard deviation u_i .

Leaf-level moments are

$$\mu_\ell = \sum_i x_{\ell,i} m_i, \quad (1)$$

$$\tau_\ell = \sqrt{\sum_i x_{\ell,i} \exp(2u_i) + \varepsilon}, \quad (2)$$

$$z_\ell = y_\ell \mu_\ell / \tau_\ell. \quad (3)$$

TreeMAPPO  Existing Methods	
Q: What is 3x2+1?	
GRPO (No Branching) Q 3x2=6. 6+1=\boxed{7} ✓ ├ 3x2=42. 42+1=\boxed{43} ⚠ should not blame the last step ├ 3x2=6, 6+1=\boxed{42} ⚠ should not blame the first step └ 3x2=42, wait, actually, 3x2=6, 6+1=\boxed{7} ⚠ should not credit the first step	Color-Advantage Mapping -1.0  +1.0
Spontaneous Branching Q OK. Let's do it step by step... ⚠ Trajectory diverges too soon ├ us do it step by step... and produce no meaningful signals └ Let's first calculate...	
Win-Rate or Heuristic-Based Credit Assignment Q 3x2=6 6+1=\boxed{7} ✓ High contrast for correctness divergence ├ 6+1=\boxed{42} ✓ High contrast for correctness divergence └ 6+1=7. Therefore, the answer is \boxed{7}. ⚠ If leaf siblings agree, the contributor is more likely parent than leaves So, 3x2+1=\boxed{7}.	
Our TreeMAPPO Algorithm Q 3x2=6 6+1=\boxed{7} ✓ High contrast for correctness divergence ├ 6+1=\boxed{42} ✓ High contrast for correctness divergence └ 6+1=7. Therefore, the answer is \boxed{7}. ✓ If leaf siblings agree, the contributor is more likely parent than leaves So, 3x2+1=\boxed{7}.	

Figure 2: Comparison of credit-assignment structures. Trajectory-level GRPO assigns one normalized outcome signal to every token; TEMPO extracts branch-gated corrections from natural shared prefixes; TreeRPO uses fixed-step tree sampling and empirical child-group rewards; TreeMAPPO combines guided token-level branching with posterior segment-credit inference.

We use a probit likelihood,

$$p(y_\ell | \{m_i, u_i\}) = \Phi(z_\ell), \quad (4)$$

where Φ is the standard normal CDF.

Our implementation initializes prior means at zero and optimizes the negative log posterior

$$\mathcal{J} = - \sum_{\ell} \log \Phi(z_\ell) + \frac{1}{2\sigma_m^2} \sum_i m_i^2 + \frac{1}{2\sigma_u^2} \sum_i u_i^2. \quad (5)$$

The current hyperparameter file uses $\sigma_m = 1.0$, $\sigma_u = 1.0$, $\varepsilon = 10^{-6}$, 120 optimization iterations, and log-standard-deviation clamps in $[-4, 2]$. Optimization is performed with gradient steps and backtracking line search, together with a numerically stable implementation of $\log \Phi$ for large negative margins.

Uncertainty-Aware Guided Branching

Posterior statistics are reused to decide where to branch next. The implementation first computes a signal-to-noise ratio for each segment,

$$r_i = \frac{m_i}{\exp(u_i) + \varepsilon}, \quad (6)$$

then standardizes these values across segments in the same tree,

$$\tilde{r}_i = \frac{r_i - \text{mean}(r)}{\text{std}(r) + \varepsilon}, \quad (7)$$

and maps them to a bounded uncertainty score,

$$s_i = \exp(-\alpha \tilde{r}_i^2), \quad \alpha = 1. \quad (8)$$

Segments whose posterior signal is close to zero receive higher branching priority because they are more uncertain.

For a branching node with parent score s_p and child scores $s_{c_1}, \dots, s_{c_b}$, the node score is

$$s_{\text{node}} = \frac{bs_p + \sum_{j=1}^b s_{c_j}}{2b}. \quad (9)$$

The final implementation-level per-token branching score also includes a model-side alternative-token term derived from the stored top- k logprob candidates. Concretely, for each candidate token position we multiply:

1. a segment or node uncertainty score,
2. a structural penalty, and
3. the relative probability of the best alternative token that differs from the already-sampled continuation.

Candidates whose best alternative has relative probability below 0.1 are discarded.

For node expansion, the structural penalty is an exponential branching-factor penalty,

$$\gamma_{\text{node}}(b) = \exp(-k_{\text{node}}(b-1)), \quad (10)$$

with $k_{\text{node}} = 0.35$ in the current implementation. For segment splitting, the structural penalty increases with the shorter side of the split relative to average trunk length, discouraging very short fragments.

From Posterior Scores to Training Advantages

After tree construction, each segment receives an unnormalized signed score proportional to $m_i / \exp(u_i)$. The methodology notes propose an additional length-aware rescaling to reduce volatility from very short reasoning spans. The generated training samples then attach token-level advantage values aligned with the input sequence.

The downstream Python training code consumes four aligned tensors per sample: `input_ids`, `labels`, `attention_mask`, and `advantages`. The loss is an advantage-weighted causal LM objective over supervised positions,

$$\mathcal{L}_{\text{train}} = \frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} \text{clip}(A_t, -c, c) \cdot \text{CE}(p_\theta(\cdot | x_{<t}), x_t), \quad (11)$$

where $\mathcal{S}$ denotes unmasked token positions and the current implementation clips A_t to $[-3, 3]$ before weighting the token cross-entropy. This is the implemented optimization interface for injecting segment-level credit into model updates.

Training-Set Construction

Because many tree leaves share prefixes, naively flattening all leaves would over-train shared segments. We therefore use a greedy de-duplication policy.

1. Score each candidate leaf trajectory by average absolute token advantage.
2. Select the strongest candidate.
3. Mark its segments as consumed.
4. Repeat among trajectories that still contain unconsumed learnable segments.
5. Aggregate selected trajectories across trees and prune the tail with a cumulative average-absolute-advantage cutoff.

Selected trajectories are finally ordered by a configurable batching policy. In length-based runs, this improves early memory stability; in by-question runs, it groups positive and negative samples from the same problem so the optimizer sees local contrasts consecutively.

Baselines and Ablations

We consider three primary comparison points.

- **Base model:** no fine-tuning.
- **GRPO-style baseline:** use 8 trunks and 8 total trajectories, yielding multiple complete rollouts without token-level branching in the main tool setup.
- **TreeMAPPO:** guided branching with MAP-based posterior credit assignment.

We also separate two ablation axes that are explicitly encoded in configs and scripts.

- **TEMPO-style branching ablation:** switch the branching policy from guided branching to spontaneous divergence-point branching while keeping posterior-based credit assignment.
- **TreeRPO ablation:** keep guided branching but replace posterior-based segment advantages with a win-rate heuristic.

In our implementation, these ablations are not merely renamed conditions: the TEMPO ablation changes `branching_policy`, while the TreeRPO ablation changes `advantage_calculation_policy`. This distinction is important because it cleanly separates exploration structure from credit estimation.

Experimental Setup

Models

We evaluate the following model-condition families.

- Qwen2.5-7B-Instruct with tool use.
- Qwen2.5-7B-Instruct without tool use.
- Qwen3-4B with tool use.
- Qwen3-4B without tool use.
- Gemma-3-4B-IT without tool use.
- Llama-3.1 and Mistral-7B without tool use.

For the main tables, tool-enabled and no-tool variants of the same backbone are treated as separate model-condition groups, since the prompting interface and action space differ.

Datasets

Training and validation are built from DeepMath, MATH, and NuminaMath.

- `hybrid_train.sqlite`: 5,000 samples per dataset, interleaved across the three in-distribution datasets.
- `hybrid_val.sqlite`: 1,000 validation samples per dataset drawn from disjoint index ranges after the training slice.

We also construct an aggregated held-out evaluation database from six datasets in this order: DeepMath, MATH, NuminaMath, AMC2023, GaoKao Math 2024, and CollegeMath. For datasets with more than 1,000 test rows, 1,000 are sampled with seed 42; for DeepMath, which lacks a dedicated test split in the current pipeline, evaluation rows are drawn from the training split after clearing the reserved training and validation examples.

Training and Evaluation Protocol

The primary experiments use a split one-shot protocol rather than a fully interleaved online reinforcement-learning loop. For each model-condition pair, we run four stages: base-policy rollout collection, training-set generation, adapter training, and checkpoint validation. Rollout and generation are performed once per condition using the base model, after which the generated trajectories and token-level advantages are reused across multiple LoRA training epochs. Validation then evaluates the base model and each trained checkpoint under the same inference protocol.

This protocol is intentionally different from a production online RL pipeline in which each epoch rolls out from the latest policy and immediately regenerates training data. The split protocol is less expensive, makes infrastructure failures easier to diagnose, and allows fair controlled comparisons among GRPO-style, TreeRPO-style, TEMPO-style, and TreeMAPPO variants under the same rollout budget.

Placeholder line chart
x-axis: total trajectories / leaves in tree (8, 16, 32)
y-axis: downstream pass@1 on validation or aggregated test
show mean and confidence interval bands

Figure 3: Branching-budget sensitivity. Replace this placeholder with a real line chart once results are available.

When compute permits, the same codebase also supports a full orchestrator mode with repeated validation rollout, training rollout collection, training-set generation, and training across policy updates; those runs are treated as confirmatory rather than the main experimental path.

The training protocol uses the following recurring settings:

- 5–10 one-shot training epochs, depending on the run budget.
- cumulative average-absolute-advantage cutoff of 0.5 for training-set retention.
- 3 optimizer iterations per training run.
- LoRA-based fine-tuning for the primary experiments.
- fixed rollout temperature 0.7.

The primary evaluation metric is pass@1 accuracy. We report aggregate accuracy together with per-dataset accuracy, and include confidence intervals when multiple independent runs are available.

Results

This section contains result placeholders to be replaced with final measured statistics. All non-result sections describe the implemented method and evaluation protocol.

Main Comparison

Table 1 is structured to compare base, GRPO, and TreeMAPPO across datasets, while grouping rows by model-condition family.

Ablation Study

The main ablation table is designed around the conditions explicitly implemented for Qwen2.5-7B with tool use: Base, GRPO, TEMPO-style spontaneous branching, TreeRPO-style heuristic credit assignment, and TreeMAPPO.

Branching-Budget Sensitivity

Our experimental plan already includes 8-, 16-, and 32-trajectory variants for the Qwen2.5-7B tool setting. This enables a direct study of how tree size affects downstream performance.

Discussion

The main conceptual benefit of TreeMAPPO is that it turns sparse trajectory-level outcomes into dense supervision without requiring reference solutions or manually annotated intermediate steps. The tree structure also allows uncertainty to be localized: early segments supported by

many mixed descendants can be recognized as uncertain, whereas a split that sharply changes success rates can receive stronger credit or blame.

Several practical design choices are worth emphasizing.

- Branch selection is not based on posterior uncertainty alone; it also depends on the model’s alternative-token probability mass, which makes branching more realistic and less likely to chase implausible continuations.
- The training loss is an advantage-weighted language-model loss rather than a clipped PPO-ratio objective. This choice keeps the optimization interface simple while preserving the central contribution: tree-based posterior credit assignment.
- Tool-enabled and no-tool settings are analyzed separately because the credit-assignment problem differs materially once trajectories can call Python and recover from arithmetic mistakes.

Limitations and Future Work

The present system has several important limitations.

- Tree construction requires more inference than flat rollout collection, so the method trades additional sampling cost for denser supervision.
- The main experiments use a split one-shot pipeline rather than fully online interleaved rollout and training. This is a practical choice driven by compute limits, queue latency, and debugging efficiency; it means the reported experiments primarily test the quality of the generated credit-assignment data, not the full closed-loop dynamics of repeated policy improvement.
- The MAP credit model intentionally uses a simple additive segment-contribution assumption. This makes inference tractable and interpretable, but it cannot represent all higher-order interactions among distant reasoning segments.
- Correctness is judged from extracted final answers, so the approach inherits the limitations of answer extraction and exact-answer comparison on mathematical benchmarks.
- The tool environment is deliberately lightweight. Richer agent tools may require additional safety checks, state tracking, and branch-selection criteria.

Conclusion

We presented TreeMAPPO, a trajectory-tree method for fine-grained credit assignment in agentic LLM training. The core idea is to branch reasoning trajectories at token granularity, infer segment-level contribution and uncertainty from mixed-success descendants, and convert those estimates into training signals for subsequent optimization. The implementation supports the central methodological ingredients, several ablations, tool/no-tool variants, and branch-budget experiments. Together, these components provide a practical route from sparse terminal rewards to localized training signal for long-form reasoning agents.

Model family	Setup	DeepMath	MATH	NuminaMath	AMC2023	GaoKao Math 2024	CollegeMath	Avg.
Qwen2.5-7B + tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B + tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B + tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B + tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen3-4B no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemma-3-4B-IT no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemma-3-4B-IT no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Gemma-3-4B-IT no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama-3.1 no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama-3.1 no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Llama-3.1 no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Mistral-7B no-tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Mistral-7B no-tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Mistral-7B no-tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 1: Main comparison across model families and datasets. Replace placeholders with measured pass@1 mean $\pm$ confidence interval.

Model family	Setup	DeepMath	MATH	NuminaMath	AMC2023	GaoKao Math 2024	CollegeMath	Avg.
Qwen2.5-7B + tool	Base	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	GRPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TEMPO-style	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TreeRPO-style	TBD	TBD	TBD	TBD	TBD	TBD	TBD
Qwen2.5-7B + tool	TreeMAPPO	TBD	TBD	TBD	TBD	TBD	TBD	TBD

Table 2: Ablation framework. The TEMPO-style condition changes the branching policy; the TreeRPO-style condition changes the segment-credit estimator.

References

Chen, W.; Ma, X.; Wang, X.; and Cohen, W. W. 2022. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *arXiv preprint arXiv:2211.12588*.

Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2022. PAL: Program-aided Language Models. *arXiv preprint arXiv:2211.10435*.

He, Z.; Liang, T.; Xu, J.; Liu, Q.; Chen, X.; Wang, Y.; Song, L.; Yu, D.; Liang, Z.; Wang, W.; Zhang, Z.; Wang, R.; Tu, Z.; Mi, H.; and Yu, D. 2025. DeepMath-103K: A Large-Scale, Challenging, Decontaminated, and Verifiable Mathematical Dataset for Advancing Reasoning. *arXiv preprint arXiv:2504.11456*.

Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *Advances in Neural Information Processing Systems*.

Huang, B.; Nguyen, T.; and Zimmer, M. 2025. Tree-OPO: Off-policy Monte Carlo Tree-Guided Advantage Optimization for Multistep Reasoning. *ArXiv preprint*.

Li, Y.; Gu, Q.; Wen, Z.; Li, Z.; Xing, T.; Guo, S.; Zheng, T.; Zhou, X.; Qu, X.; Zhou, W.; Zhang, Z.; Shen, W.; Liu, Q.; Lin, C.; Yang, J.; Zhang, G.; and Huang, W. 2025. TreePO:

Bridging the Gap of Policy Optimization and Efficacy and Inference Efficiency with Heuristic Tree-based Modeling. *arXiv:2508.17445*.

Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let’s Verify Step by Step. *arXiv preprint arXiv:2305.20050*.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. *arXiv:1707.06347*.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

Tran, H.; Yao, Z.; and Yu, H. 2025. Exploiting Tree Structure for Credit Assignment in RL Training of LLMs. TEMPO, *arXiv:2509.18314*.

Wang, X.; Ma, Z.; Wang, Y.; Ji, Y.; Yang, S.; Chen, G.;

Wang, P.; and Chu, X. 2026. APPO: Agentic Procedural Policy Optimization. arXiv:2606.12384.

Yang, Z.; Guo, Z.; Huang, Y.; Liang, X.; Wang, Y.; and Tang, J. 2025. TreeRPO: Tree Relative Policy Optimization. arXiv:2506.05183.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*.